Computational Statistics (1999) 14:375–395

Computational Statistics

© Physica-Verlag 1999

Adaptive proposal distribution for random walk Metropolis algorithm

Heikki Haario[1], Eero Saksman[1] and Johanna Tamminen[2]

[1] Department of Mathematics, P.O.Box 4 (Yliopistonkatu 5), FIN-00014 University of Helsinki, Finland
[2] Finnish Meteorological Insitute, Geophysical Research Division, P.O.Box 503, FIN-00101 Helsinki, Finland

Summary

The choice of a suitable MCMC method and further the choice of a proposal distribution is known to be crucial for the convergence of the Markov chain. However, in many cases the choice of an effective proposal distribution is difficult. As a remedy we suggest a method called Adaptive Proposal (AP). Although the stationary distribution of the AP algorithm is slightly biased, it appears to provide an efficient tool for, e.g., reasonably low dimensional problems, as typically encountered in non-linear regression problems in natural sciences. As a realistic example we include a successful application of the AP algorithm in parameter estimation for the satellite instrument 'GOMOS'. In this paper we also present systematic performance criteria for comparing Adaptive Proposal algorithm with more traditional Metropolis algorithms.

Keywords: MCMC, Adaptive MCMC, Metropolis–Hastings algorithm, convergence.

1 Introduction

In this paper we will introduce a new *adaptive* Markov Chain Monte Carlo method that yields a quick and flexible tool for estimating e.g. posteriori distributions in parameter estimation problems. Basically the method consists of the well-known Metropolis algorithm (Metropolis, Rosenbluth, Rosenbluth, Teller & Teller 1953) (Hastings 1970), where however, the proposal distribution is tuned along the search according to the covariance calculated from a fixed number of previous states. This procedure makes sure that the MCMC process adapts to the target distribution, at least locally, and thus keeps the search effective all the time. We call the method *Adaptive Proposal* (AP). For other adaptive approaches and related work see (Gilks, Best & Tan 1995), (Gilks & Roberts 1995), (Gilks, Roberts & George 1994), (Gilks, Roberts & Sahu 1998), (Gilks & Wild 1992), (Sahu & Zhigljavsky 1998a) and (Sahu & Zhigljavsky 1998b).

This paper is devoted to investigating properties and practical usefulness of the Adaptive Proposal algorithm. On the basis of our results we believe that the method will provide a valuable tool in various respects:

- The Metropolis algorithm or one of the variants, especially the single component Metropolis–Hastings algorithm, are often used in situations where the a priori knowledge of the target distribution is quite limited. In such situations the selection or 'tuning' of the proposal distribution may be the bottleneck of the whole work – see Section 6 for a real example. The Adaptive Proposal solves this problem by utilizing the information obtained during the simulation and, thus, automatically adapting the proposal distribution to the target distribution.

- The Adaptive Proposal does not simulate exactly the target distribution. However, in practice it is very close to be exact. Our tests show that the difference of the limit distribution of AP and the true target distribution is negligible in most, reasonably behaving cases. Therefore, AP provides an easy to use MCMC algorithm for various practical purposes. Moreover, AP may be applied as an *adaptive burn in* since it may be used to produce an efficient Gaussian (i.e. multinormal) proposal distribution for the standard Metropolis algorithm.

- In applications with a non-linear, curved posteriori distribution there apparently does not exist a simple and effective proposal distribution. In such situations our tests indicate that AP finds the true posteriori region more effectively than a Metropolis search with a fixed, reasonably well selected proposal distribution.

- Especially, if there is correlation between the components, our tests clearly demonstrate how AP is superior to the single component Metro-

polis-Hastings algorithm or Metropolis algorithm where the proposal distribution is not properly tuned. Moreover, it is not essentially inferior to the Metropolis algorithm with an optimally chosen proposal distribution.

There is a great variety of different MCMC methods. The two major approaches are the Gibbs sampling (Gelfand & Smith 1990) and the standard Metropolis–Hastings methods (Metropolis et al. 1953), (Hastings 1970). Since the critical features are different for the two methods, we will concentrate in this paper only on the Metropolis–Hastings methods and discuss the problems of selecting an effective proposal distribution. Hence the Adaptive Proposal algorithm is compared here only with the methods closely related to it. A more comprehensive comparison with other MCMC methods including Gibbs type methods and other adaptive methods will be a topic of further work.

The contents of the present paper is organized as follows. In Section 2 we discuss the difficulties that arise in selecting an effective proposal distribution for Metropolis–Hastings methods. Section 3 contains a detailed definition of the Adaptive Metropolis algorithm. In Section 4 we make a detailed comparison between AP and the commonly used Metropolis methods. As the test targets we have used both correlated and uncorrelated Gaussian distributions, and two non-linear, 'banana shaped', Gaussian distributions. The approximate ergodic properties of the AP algorithm are discussed in Section 5.

We have also applied the Adaptive Proposal algorithm in a more realistic inverse problems application. This problem arises in the data processing of the GOMOS (Global Ozone Monitoring by Occultation of Stars) satellite instrument. In this interesting application each analysis requires about 50 inversions. Moreover, when applying the Metropolis–Hastings method each inversion requires a quite individual proposal distribution. It appears that Adaptive Proposal provides a valuable tool for this kind of applications.

2 Pitfalls in selecting the proposal distribution

The shape and the size of the proposal distribution $q(\cdot)$ is known to be very crucial for the convergence of the Markov chain corresponding to the MCMC algorithm, see for example (Gelman, Roberts & Gilks 1996), (Gilks, Richardson & Spiegelhalter 1995), (Gilks et al. 1998) and (Roberts, Gelman & Gilks 1994). Even when the acceptance probability does not depend on the proposal distribution (like in the Metropolis algorithm) the mixing of the chain depends on it. It is well known that when the proposal distribution is too large, too many candidate points are rejected and therefore the chain

covers slowly the target distribution. On the other hand, when the proposal distribution is too small, too many candidate points are accepted and the chain converges slowly again. In addition, to ensure computational efficiency one should choose the proposal distribution so that sampling from it would be fast and easy.

In (Gelman et al. 1996) and (Roberts et al. 1994) the authors present optimal acceptance rates for Gaussian target distributions in different dimensions when the proposal distribution is also Gaussian. Based on this study they propose to monitor the acceptance rate (i.e. ratio of accepted proposal points over all points) of the chain and scale the proposal distribution with respect to the optimal acceptance rate. This method is widely used in practice.

However, the tuning of the size and the shape of the proposal distribution can be time consuming and tricky since the acceptance rate does not take into account the shape of the underlying target distribution with respect to different parameters. When the parameters correlate or are of different size the tuning can be specially difficult.

The proposal distribution can be tuned more easily in single component Metropolis algorithm since for each component we get an individual acceptance rate value. However, it is well known that this method converges slowly to the target distribution if there are correlations between different components.

3 Adaptive Proposal

To avoid the difficulties of tuning the proposal distribution in Metropolis–Hastings methods, as discussed in the previous section, we propose a method called here Adaptive Proposal (AP).

We now give a detailed description of the Adaptive Proposal method. The basic idea is to update the proposal distribution with the knowledge we have so far learned about the target distribution. Otherwise the algorithm may be viewed as a Metropolis process where the proposal distribution q depends on time. Thus, assume that the points $X_1, \ldots, X_k$ are already sampled. Then a candidate point Y is sampled from the proposal distribution $q_k(\cdot|X_1, \ldots, X_k)$ which now depends on a part of the history $(X_{k-H+1}, \ldots, X_k)$. The candidate point is either accepted or rejected with the probability

$$\alpha(X_k, Y) = \min\left(1, \frac{\pi(Y)}{\pi(X_k)}\right), \tag{1}$$

where $\pi(\cdot)$ denotes the (unscaled) probability density of the target distribution. In the case of acceptance we set $X_{k+1} = Y$, otherwise $X_{k+1} = X_k$. As the proposal distribution $q_k(\cdot|X_1, \ldots, X_k)$ we have applied a Gaussian dis-

tribution with mean X_k and covariance, that depends on a part of the the history.

Thus the crucial thing is how the proposal distribution depends on the history. In AP this has been solved in the following manner: Assume that the target distribution $\pi(\cdot)$ is d-dimensional, i.e. it is defined on a subset of $\mathbb{R}^d$. Suppose then that at time t we have sampled at least H points $\{X_1, \ldots, X_{t-H+1}, \ldots, X_t\}$. Here the fixed integer H is the *memory* parameter. The proposal distribution q_t for sampling proposal state Y is chosen as follows

$$q_t(\cdot|X_1, \ldots, X_t) \sim N(X_t, c_d^2 R_t),$$

where R_t is the $d \times d$ - covariance matrix determined by the H points $X_{t-H+1}, \ldots, X_t$ and the scaling factor c_d depends only on the dimension d (the role of c_d is discussed later in this section).

The covariance R_t may be calculated by collecting the points $X_{t-H+1}, \ldots, X_t$ in a $H \times d$ matrix K, where each row represents one sampled point. Then

$$R_t = \frac{1}{H-1} \widetilde{K}^T \widetilde{K}. \tag{2}$$

where we denote by $\widetilde{K}$ the centered matrix $\widetilde{K} = K - \mathrm{E}[K]$. In practice an easy way to make a sample from $N(X_t, c_d^2 R_t)$ is, for instance, to observe that

$$N(X_t, c_d^2 R_t) \sim X_t + \frac{c_d}{\sqrt{H-1}} \widetilde{K}^T N(0, \mathrm{I}_H),$$

where $N(0, \mathrm{I}_H)$ is the normalized H-dimensional Gaussian distribution.

In this way we obtain samples that adapt to the 'current' situation of the MCMC search, see Figure 1. The use of the scaling parameter c_d is heuristical, but its usefulness is supported by empirical tests. As a basic choice, we have adopted the value $c_d = 2.4/\sqrt{d}$ from (Gelman et al. 1996), which corresponds to theoretically optimal mixing properties of the Metropolis-Hastings search in the case of Gaussian targets and Gaussian proposals.

In computations reported in the present paper we have not updated the covariance of the proposal distribution at each step of the process, but rather kept it fixed for U steps. Thus in this case the R_t is updated only at times that are multiples of U and kept fixed in between. Here the parameter U is called *the update frequency*. The practical significance of the parameters H and U is discussed later in Section 4.4.

In order to avoid a slow start of the algorithm it is possible to employ special tricks. Naturally, if a priori knowledge (like the maximum likelihood point or approximate covariance of the distribution) is available it can be utilised in the start up of the process.

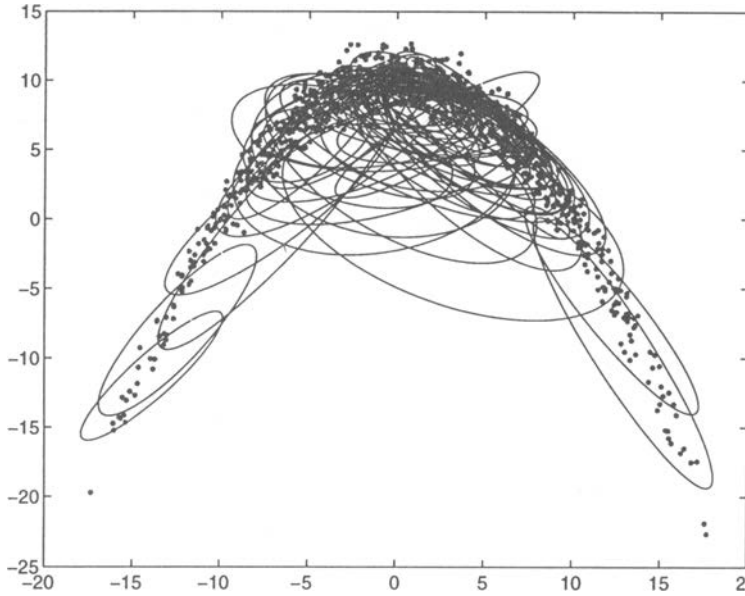

Figure 1: An example of the simulated chain when the target function was 2-dimensional 'banana-shaped' distribution. The ellipsoids refer to 95% probability regions of different proposal distributions. In this example the tuning parameters were $H = 200$ and $U = 200$.

Without any a priori knowledge we can employ the *greedy start* procedure: during a short initial period one updates the proposal using only the accepted states. Moreover, during this period the memory H and the update frequency U can be chosen smaller than usually. Afterwards the AP is run as described above. During the early stage of the chain, it is natural to require that the chain moves at least a little. If it has 'not moved enough', the proposal distribution could be shrinked by a constant factor.

4 Comparing Metropolis algorithms and Adaptive Proposal algorithm

In this section we report the computer simulations we have performed in order to compare two basic MCMC methods and the Adaptive Proposal algorithm. The methods compared are

- *Single component Metropolis algorithm* (abbreviated here by SC) with Gaussian (i.e. normal) proposal distributions. This is the original Metropolis algorithm (Metropolis et al. 1953) based on sampling each

component of the new state separately one after another.

- *Metropolis algorithm* (M) with a Gaussian (i.e. multinormal) proposal distribution. This is a modification of the single component Metropolis algorithm, where now all the components are sampled at the same time.
- Adaptive Proposal algorithm (AP).

4.1 The test target distributions

Our purpose is to study the goodness of the simulation in a comprehensive manner. The test distributions are constructed so that it is easy to compute, e.g. different regions of probability. This enables us to study how these probability regions are covered by the chains. The different target distributions which are presented in Figure 2 are

- Uncorrelated Gaussian distribution (π_1)
- Correlated Gaussian distribution (π_2)
- Moderately non-linear 'banana shaped' distribution (π_3)
- Strongly non-linear 'banana shaped' distribution (π_4)

The uncorrelated Gaussian test target π_1 is a centered multivariate normal distribution $N(0, C_1)$, with the covariance C_1 =diag$(100, 1, \ldots, 1)$. Thus the shape of it is an ellipsoid with one semi-axis ten times the size of the others. The correlated Gaussian test target π_2 is just the distribution π_1 rotated in such a way that the long semi-axis corresponds to the direction $(1, 1, \ldots, 1)$.

The non-linear banana-shaped distributions are constructed from the Gaussian ones by 'twisting' them as follows. Let f be the density of the multivariate normal distribution $N(0, C_1)$ with the covariance again given by C_1 =diag$(100, 1, \ldots, 1)$. The density function of the 'twisted' Gaussian with the nonlinearity parameter $b > 0$ is given by

$$f_b = f \circ \phi_b,$$

where the function ϕ_b is

$$\phi_b(x) = (x_1, x_2 + bx_1^2 - 100b, x_3, \ldots, x_n).$$

Thus ϕ_b changes only the second coordinate and the determinant of the Jacobian of ϕ_b is identically equal to 1. This makes it very easy to calculate the probability regions for the twisted Gaussian targets. By definition the twisted target function is centered.

The nonlinearity of function ϕ_b increases with b. In our tests we have applied the value $b = 0.03$ for moderately twisted target π_3, and $b = 0.1$ for strongly twisted target π_4.

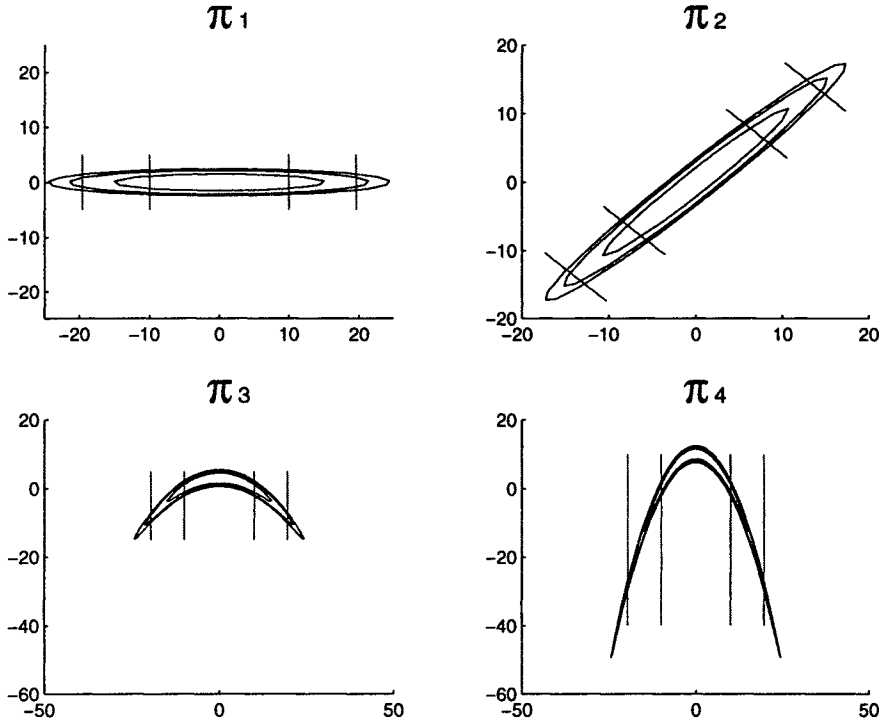

Figure 2: Test target distributions $\pi_1, \ldots, \pi_4$ in dimension 2. The contours refer to 68.3%, 90% and 95% probability regions. The lines separate the 1-dimensional probability region between 68.3% and 95%.

4.2 Performing the test runs

Each test was replicated 100 times. The starting point (if not otherwise specified) for each repetition was chosen randomly relatively close to the mean of the target. As the distribution from which we sampled starting points we used Gaussian distribution $N(\overline{x}, \mathrm{I}_d)$ where the mean $\overline{x}$ is the expectation of the target distribution and I_d is the d dimensional unit vector.

The number of target function evaluations was the same for each method, but it varied according to the dimension and the test target distribution used. Note that the chain length in the single-component Metropolis method is therefore smaller than in the other two methods, but the computing time is about the same. The burn in was always half of the chain length for simplicity.

We have tried to be fair in choosing the fixed proposal distributions for the single component Metropolis and Metropolis algorithms. The proposal distributions for the SC algorithm in the case of the uncorrelated Gaussian target π_1 are chosen to be the (fairly optimal) one dimensional normal distributions $Q_i \sim N(0, C_1(i,i))$ for $i = 1, \ldots, d$. In the case of the correlated Gaussian targets a reasonable choice has been $Q_i \sim N(0,1)$ for each i. In the case of Gaussian target distributions π_1 and π_2 the Metropolis algorithm was run using Gaussian proposal distributions with known covariances C_1 and C_2 together with the optimal scaling (Gelman et al. 1996), i.e. the proposals were $(2.4/\sqrt{d}) \times N(0, C_1)$ and $(2.4/\sqrt{d}) \times N(0, C_2)$, respectively. Thus the algorithm should perform optimally. In the case of the non-linear targets we have used the corresponding non-twisted Gaussian proposal distribution $(2.4/\sqrt{d}) \times N(0, C_1)$.

The Adaptive Proposal algorithms were run with various values of update and frequency parameters depending on the dimension and the target distribution. More details on the selection are given in Section 4.4 However, we already want to emphasize that the AP is not especially sensitive on these parameters. We note that in all our test cases the Adaptive Proposal algorithm is used in a fully adaptive way (i.e. it has not been used as an adaptive burn in) and we have employed the greedy start procedure that was explained in Section 3.

4.3 Results of the tests

As usual in the MCMC simulations we have calculated the expectation values for the components. This vector information is condensed by computing the norm of the error from the true value, which is origin in our case. However, even if the expectation value is approximately correct there might still be a considerable error in the simulation of the target distribution. In order to see how well the chain actually covers the target distribution we have computed the percentage of sampled points that hit a given probability region. Moreover, in order to study how well the extreme tails of the distributions are simulated we have projected the samples to the 1-d direction of the main semi-axis, and compared this distribution to the correct one. We have performed the tests in different dimensions between 2 and 32 and include here some tables of the results in dimension 8. More comprehensive test results are tabulated in (Haario, Saksman & Tamminen 1998a). Here we first explain how to read the tables.

Each number in the tables is an average value over 100 repetitions. The rows of the tables give the performance criteria while the columns represent different methods compared. Here is the list of performance criteria used in the tables:

- mean($||E||$) denotes the mean distance of the expectation values from the true value at origo calculated over 100 repetitions: mean($||E||$) $= \frac{1}{100}\sum_{j=1}^{100}\left(\sum_{i=1}^{d}(E_j^i)^2\right)^{\frac{1}{2}}$, where E_j is the mean vector of chain j.

- std($||E||$) denotes the standard deviation of the distances from the true value at origo.

- err($\leq$ 68.3%) is the mean error of the percentages of the sampled points that hit the probability region inside 68.3% level.

- std($\leq$ 68.3%) is the standard deviation of the percentages of the sampled points that hit the probability region inside 68.3% level.

- err($\geq$ 99%) and std ($\geq$ 99%) are, analogously, the mean error and the standard deviation of the percentages of points that hit the probability region outside 99%.

- 1-d err(68.3-95%) is the mean error of the percentages of the sampled points that hit the 1-dimensional probability region between 68.3% and 95%. The 1-dimensional probability region contains all those points whose projections to the direction of the main semi-axis of the target distribution belong to the corresponding 1-dimensional probability interval. In the case of the non-linear target distributions π_3 and π_3 the main semi-axis is $[1, 0, \ldots]$ (see Figure 2).

- 1-d std(68.3-95%) is the standard deviation of the percentages of the sampled points that hit the 1-dimensional probability region between 68.3% and 95%.

- 1-d err(95-99%) and 1-d std(95-99%) values are the corresponding ones between 95% and 99% limits.

- acc. rate is the percentage of the accepted proposal states.

For convenience, we summarize here the results of our tests in dimension 8 graphically. In the top picture in Figure 3 we show the mean error and the standard deviation of the percentages of the sampled points that hit the probability region inside 68.3% limit (i.e. err($\leq$ 68.3%) and std($\leq$ 68.3%)) for all the targets $\pi_1, \ldots, \pi_4$. The dots refer to the mean error value and the error-bar to the standard deviation. The correct value is denoted with a horizontal line. In the lower picture in Figure 3 we present in a similar way the mean error and standard deviation of the percentages of the sampled points that hit the 1-dimensional probability region between 68.3% and 95% (i.e. 1-d err($68.3 - 95\%$) and 1-d std($68.3 - 95\%$)). The true percentage 26.7% is denoted with a horizontal line.

Table 1: 8-dimensional non-linear distributions. Each number is calculated from 100 repetitions. Length of the burn-in was 50% of the length of the chain.

	Non-linear (π_3)			Non-linear (π_4)		
Method	**SC**	**M**	**AP**	**SC**	**M**	**AP**
Function evaluations	**40000**	**40000**	**40000**	**80000**	**80000**	**80000**
mean($\|E\|$)	2.40	2.64	1.31	6.53	7.89	4.85
std($\|E\|$)	1.79	2.81	0.72	4.79	7.54	4.20
err($\leq$ 68.3%)	1.30	0.18	0.80	2.46	0.35	2.13
std($\leq$ 68.3%)	4.59	6.70	2.92	6.48	9.79	5.34
diff($\geq$ 99%)	0.16	0.03	0.01	0.27	0.07	0.14
std($\geq$ 99%)	0.40	0.66	0.62	0.34	0.97	0.45
1-d err(68.3-95%)	1.26	0.57	0.53	0.67	0.45	3.07
1-d std(68.3-95%)	7.64	4.98	3.14	24.84	25.13	7.81
1-d err(95-99%)	0.83	0.52	0.60	3.07	1.49	1.76
1-d std(95-99%)	5.89	7.58	1.99	5.33	11.34	6.36
acc. rate (%)	70.37	15.19	14.57	70.39	7.84	9.59

Figure 3 shows that the uncorrelated Gaussian target π_1 is simulated roughly equally well by the different methods employed. Note that the AP algorithm is not inferior to the ideally tuned Metropolis algorithm. The figure also indicates that in the case of the correlated Gaussian target π_2 the single component Metropolis algorithm is clearly weaker than the other two methods. In this case the optimally tuned Metropolis algorithm and the Adaptive Proposal algorithm give similar results again.

In the moderately twisted Gaussian case π_3 it turns out that, in fact, the AP algorithm gives more reliable results than even the Metropolis with a well chosen proposal distribution. This effect is even more clear in the strongly twisted case π_4. Especially the far ends of the strongly twisted target distribution is more accurately covered by the AP algorithm than by the other methods.

Note also the characteristic features in cases π_3 and π_4: the mean values obtained by the AP algorithm differ from the true one more than that by the Metropolis algorithm, while the uncertainty (std) is much larger with the Metropolis algorithm. Presumably this can be partially attributed to the fact that the AP is an approximate method and thus the simulation may be slightly biased. The better mean value with the Metropolis algorithm is due to the exact ergodicity of the method and the large number of chains (100 repetitions) from which the mean is computed. However, we are interested in

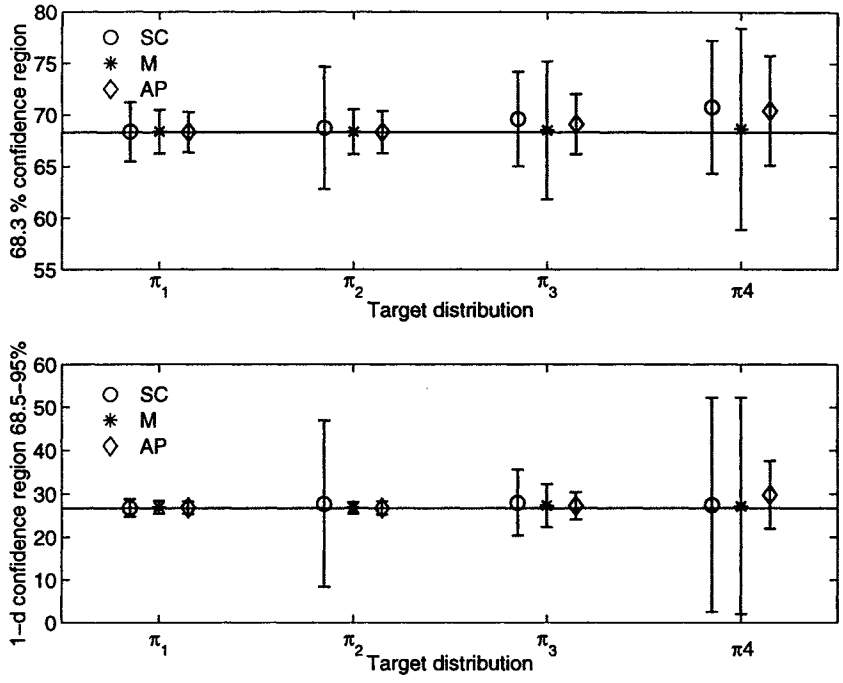

Figure 3: The results with different 8-dimensional target distributions. The top figure represents the mean and the standard deviation of the errors for 68.3% probability limit. The lower figure represents the mean and the standard deviation of the errors for the probability region between 68.3% and 95%.

how the methods work, in average, with a single chain of reasonable length. In this respect the standard deviation more properly reflects the average goodness of the results.

The results are similar in other dimensions and the corresponding tables can be found in (Haario et al. 1998a). In higher dimensional spaces the adaptation of the AP algorithm is naturally slower which means that the burn-in period increases.

We have also compared the AP algorithm and the Metropolis algorithm with a reasonable proposal distribution, that might be considered as a typical choise in Metropolis runs: a spherically symmetric Gaussian distribution whose size is tuned according to the acceptance rate. The results, with the target distribution π_1 and $0.93 \times N(0, \mathrm{I}_8)$ as the proposal distribution for the Metropolis algorithm, are presented in Table 2. On the basis of the acceptance rate alone one might consider the proposal distribution for Metropolis algorithm to be well chosen. However, as the table shows the

Table 2: 8-dimensional Gaussian target distribution π_1. The Metropolis algorithm was run with a non-ideal Gaussian proposal distribution.

Dimension	M	AP
Function evaluations	20000	20000
mean($\|\|E\|\|$)	2.96	0.46
std($\|\|E\|\|$)	2.31	0.33
diff($\leq$ 68.3%)	0.23	0.02
std($\leq$ 68.3%)	4.40	1.95
diff($\geq$ 99%)	0.01	0.03
std($\geq$ 99%)	0.61	0.32
acc. rate (%)	27.72	27.96

Table 3: 8-dimensional correlated Gaussian target distribution π_2. In the second case the AP algorithm was run using starting values close to $x_0 = [100, \ldots, 100]^T$.

	AP	AP
Starting values	$N(0, \mathrm{I}_8)$	$N(x_0, \mathrm{I}_8)$
Function evaluations	20000	20000
mean($\|\|E\|\|$)	0.42	0.47
std($\|\|E\|\|$)	0.25	0.31
diff($\leq$ 68.3%)	0.03	0.08
std($\leq$ 68.3%)	2.04	2.39
diff($\geq$ 99%)	0.01	0.01
std($\geq$ 99%)	0.41	0.39
acc. rate (%)	27.85	28.01

results of the Metropolis algorithm are clearly worse than the results of the AP method.

The effect of selecting the starting values are demostrated in Table 3. In this case the target distribution was the correlated Gaussian distribution π_2. As the starting values for AP we used values close to the origin and close to $[100, \ldots, 100]^T$. We note that the relatively badly chosen starting values, in the second case, do not essentially effect the results here.

The updating procedure of the AP algorithm naturally somewhat increases the CPU time as compared to the Metropolis algorithm, especially if the dimension is high. However, in a great majority of practical applications most

of the CPU time is spent in evaluating the objective function. According to our tests AP yields, for a given number of function evaluations, better results than the Metropolis algorithm with a non-ideal proposal distribution. Hence, in realistic examples AP more likely decreases the overall CPU time required for reliable results.

4.4 Insensitivity with respect to the tuning parameters

We have studied with similar test procedures as above how the values of the memory parameter H, the update frequency U and the scaling parameter c_d influence the performance of the AP algorithm (see (Haario et al. 1998a)). It turns out that all three variables do have an effect, but *the dependence on U and H is rather weak.* Instead, a proper scaling by c_d is more essential. The choice $c_d = 2.4/\sqrt{d}$ (discussed in (Gelman et al. 1996)) clearly improves the results in the case of Gaussian targets, if compared to the runs with no scaling.

A quadratic regression model, with H and U as the independent variables, was found to adequately explain the response variables tested: the ratio of visits inside the 68.3% region and outside the 99% region. The response variables were taken as averages over 100 repetitions. Figure 4 exhibits a summary of the runs for the probability regions in the form of the response surface. The surface clearly shows that the method is quite insensitive of the values of H and U, as soon as they are selected to be roughly of the correct order of magnitude. Indeed, with a smaller number of repetitions for the averages, the factors H and U turned out to be statistically insignificant in the range shown in the pictures.

As usual in MCMC simulations, the total chain length N must increase for larger space dimensions d. At the same time, the optimal dimensions of H and U also increase. Because of the observed insensitivity we limit here ourselves to presenting 'working' values for parameters H and U in different dimensions. The total length of the chains N is assumed to be 'sufficient' for each d. Table 4 lists for the Gaussian case some representative working values for H and U for the dimensions d shown.

To study the effects of nonlinearities we used the 'twisted' Gaussian distributions π_3 and π_4. Again, the results are quite insensitive with respect to parameters H and U as soon as they are not too small. Our tests indicated that as compared with the linear case, we should choose a little larger values for H and U. For example, in dimension 8 we used for moderately non-linear target parameter values $H = U = 1600$ and for strongly non-linear target $H = U = 2000$.

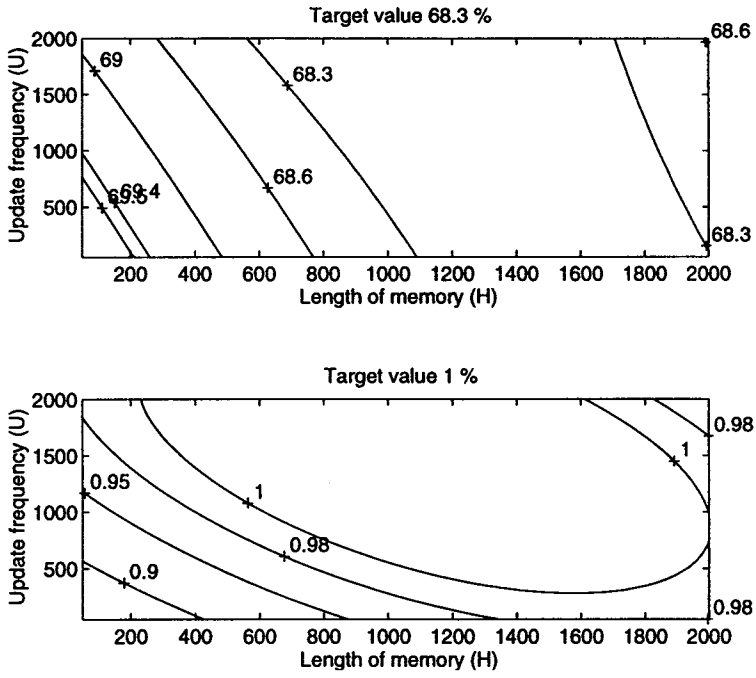

Figure 4: Sensitivity of Adaptive Proposal algorithm on memory and update frequency parameters. Target function is 8-dimensional Gaussian.

Table 4: Reasonable working values for memory and update frequency parameters for Gaussian targets.

Dimension	Chain length	History H	Update frequency U
2	5000	200	200
4	10000	400	400
8	20000	700	700
16	40000	1200	1000
32	80000	2700	2000

5 Ergodic properties of the Adaptive Proposal

In this section we consider shortly the convergence properties of the AP process (X_n). For simplicity we assume that the target distribution π is bounded and we assign a lower bound for the size of the proposal distribution. This makes sure that the algorithm visits almost surely every set of positive

π-measure. Actually, our tests suggest that the mobility of the AP is superior to e.g. the Metropolis algorithm, which is an expected consequence of the adaptive nature of the AP.

However, the Adaptive Metropolis does not yield unbiased simulation of the target distribution π. As an example, let us consider the case where the update frequency $U = 1$. In order to determine the actual distribution yielded by the algorithm one may consider the chain $Y_k = (X_k, X_{k-1}, \ldots, X_{k-H+1})$ in the state space $\mathbb{R}^{d \times H}$. Now the general theory of ergodicity for Markov processes becomes applicable. By projecting the limit distribution of the chain (Y_k) back to $\mathbb{R}^d$ to obtain the distribution $\tilde{\pi}$ that (X_k) eventually simulates. Thus for measurable sets A it holds almost surely that

$$\tilde{\pi}(A) = \lim_{n \to \infty} \frac{1}{n}(\chi_A(X_1) + \chi_A(X_2) + \ldots + \chi_A(X_n)),$$

where χ_A is the characteristic function of set $A \subset \mathbb{R}^d$.

According to the results of the tests we conclude that the difference between π and $\tilde{\pi}$ appears not to disturb the simulations within the class of target distributions used in the tests. In fact, the overall accuracy of the AP algorithm is comparable to the accuracy of a standard Metropolis algorithm with a well chosen proposal distribution.

However, if the target distribution has certain specific properties the difference between π and $\tilde{\pi}$ may play a role. This is demonstrated by the following example, where the two-dimensional distribution π has a very high density on a strip.

Example. Let us consider a density f defined on the rectangle $R = [-18, 18] \times [-3, 3] \subset \mathbb{R}^2$ containing the middle strip $S = [-0.5, 0.5] \times [-3, 3]$. We set

$$f(x) = \begin{cases} 36 & \text{if } x \in S \\ 1 & \text{if } x \in R \setminus S. \end{cases}$$

With this choice $\pi(S) : \pi(R \setminus S) = 36 : 35$ and hence about half of the mass is concentrated on S. Thus the AP algorithm stays about the same time on S and on R. However, S and R are thin rectangles with opposite direction which forces the AP to regularly turn around the direction of the proposal distribution. This causes notable bias in the simulation on S, see Figure 5. In fact, the relative error in the simulation on S is 13%. There is also a slight error in the simulation near the far ends of the rectangle R.

To achieve ergodicity we could simply fix the proposal distribution at some stage. The process then becomes the standard Metropolis method. More theoretical aspects of the ergodicity properties of variants of the AP algorithm are discussed in the preprint (Haario, Saksman & Tamminen 1998b).

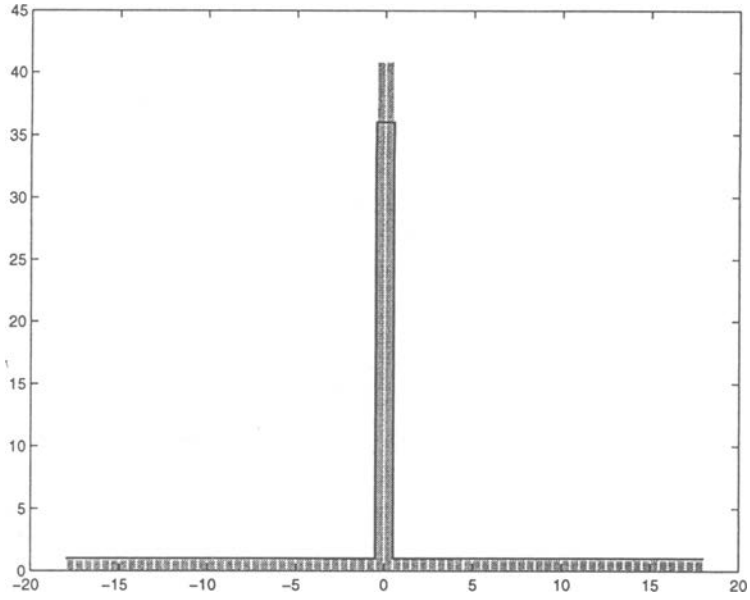

Figure 5: The solid line represents the 1-dimensional projection of true target distribution of Example 1. The histogram shows the mean on 100 runs of the AP with 100000 states. The starting point has been random, but close to the origin.

6 Simulated GOMOS example

We will here discuss as a more realistic (but still simulated) example an inversion problem which rises in the GOMOS data processing. The GOMOS (Global Ozone Monitoring by Occultation of Stars) instrument is a European Space Agency's satellite instrument which will measure atmospheric gases, and especially ozone in the stratosphere.

The instrument measures transmission data $T_z^{obs}(\lambda_i)$ at wavelengths λ_i, ($i = 1, \ldots, m \approx 1400$) repeatedly at altitudes z between 20 and 100 km. The transmission term describes how much stellar light is absorbed in the atmosphere and it is proportional to the amount of different gases in the atmosphere. The physical dependence of transmission T_z on the amount of gases is given by

$$T_z(\lambda_i) = e^{-\sum_{j=1}^{J} N_j(z)\alpha_j(\lambda_i)},$$

where the unknown parameters $N(z) = [N_1(z), \ldots, N_J(z)]$ are the horizontally integrated densities of different gases at altitude z, called the *line densities*, α_j are known model parameters and the sum is over the number of

gases J.

Assuming normally distributed non-correlated measurement noise with mean zero and diagonal covariance matrix $C = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_m^2)$, the likelihood function at altitude z reads

$$\mathrm{P}(T_z^{obs} \mid N(z)) = \frac{1}{(2\pi)^{\frac{n}{2}}|C|^{\frac{1}{2}}} e^{-\frac{1}{2}S(N(z))},$$

where

$$S(N(z)) = \sum_{i=1}^{m} \frac{1}{\sigma_i^2} \left(T_z(\lambda_i) - T_z^{obs}(\lambda_i)\right)^2 .$$

For more information of the data processing of the GOMOS instrument see (Kyrölä, Sihvola, Kotivuori, Tikka, Tuomi & Haario 1993).

The characteristic feature here is that each measurement, in fact, consists of a set of some 50 data sets obtained at different altitudes. The parameter estimation is performed separately for each data set. The size of the Bayesian posteriori distribution depends strongly on the measurement altitude and also on the brightness of the star. It turned out that any given fixed proposal distribution did not work for all the altitudes and the individual tuning of the proposal distribution at each altitude was difficult and laborious. On the contrary, a successful implementation of the AP algorithm was effortless.

To get an idea how well the AP algorithm works in the GOMOS case we have compared the point estimates computed with AP in a case where we know the true values. As the point estimates of the AP algorithm we used the mean values of the computed MCMC chain. We repeatedly performed the computation at $K = 58$ altitudes between 18 and 90 km. We also repeated the simulations $n = 50$ times at each altitude varying only random noise. As a comparison we have performed the same inversion with the traditional Levenberg-Marquardt method (Press, Teukolsky, Vetterling & Flannery 1992) which is based on searching for the maximum likelihood solution, the AP algorithm with non-informative prior and the AP algorithm with a priori positive gas densities.

The results are presented in Figure 6 where we have compared the statistical error values for retrieving ozone line density. The relative error corresponding each altitude z_k $(k = 1, \ldots, K)$ is calculated in the following way:

$$err(z_k) = \sqrt{\sum_{i=1}^{n} \frac{1}{n} \left(\frac{N^i(z_k) - N^{true}(z_k)}{N^{true}(z_k)} \right)^2} \times 100\%, \qquad (3)$$

where $N^i(z_k)$ is the ith point estimate of the ozone line density at altitude z_k. Since we are using simulated data we know the true value $N^{true}(z_k)$.

We note in the Figure 6 that at high and low altitudes where the signal-to-noise ratio is poor the AP algorithms give more accurate results. At low

altitudes the Levenberg-Marquardt method obviously does not find a good maximum likelihood solution whereas the expectation values, computed with the AP algorithm are better point estimates. The requirement of positive densities clearly improves the accuracy specially at high altitudes. The figure indicates that the AP method gives slightly better point estimates than the traditional parameter estimation method.

Moreover, Figure 6 also shows that the AP algorithm works reliably in the GOMOS example. The method was fully automatic and we used the same initial proposal distribution for all the altitudes even though the size of the posteriori distributions and the magnitudes of the point estimates varied a lot. As the starting points for the AP algorithm we used values close to the origin, which is a relatively bad choice. The AP algorithm seems to have found the target distribution and adapted the proposal distribution correctly.

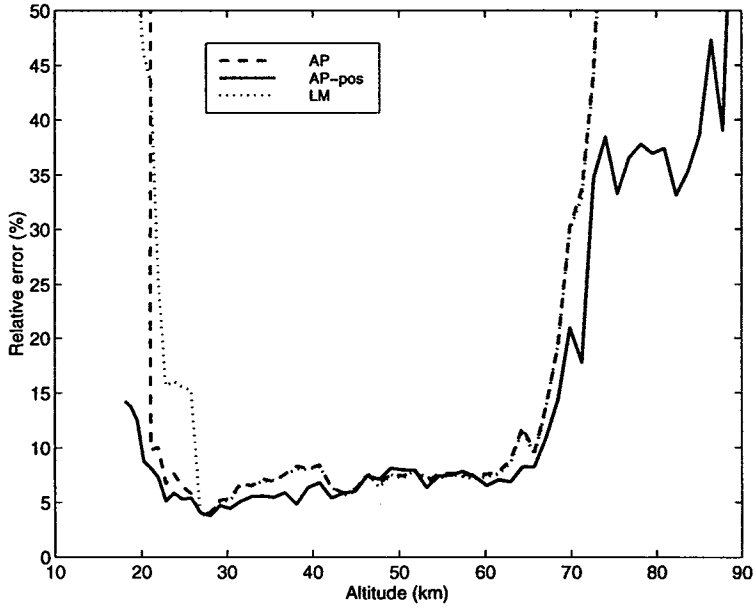

Figure 6: Retrieval error (%) of the ozone line density as a function of measurement altitude. The dashed line refers to the expectation values of the Adaptive Proposal method with non-informative prior. The continuous line corresponds to the expectation values of the AP method with the requirement of positive densities. The total length of the chain was 20000 and burn-in was 10000 in both cases. The dotted line corresponds to the maximum likelihood estimation (with the Levenberg–Marquardt routine).

7 Discussion

In this paper we have introduced a new MCMC method called the Adaptive Proposal (AP). It is a variation of the Metropolis algorithm where we update the proposal distribution as the chain learns more about the target distribution. The advantage of the AP algorithm is that we avoid a troublesome tuning of the proposal distributions.

Our tests indicate that the Adaptive Proposal algorithm, although not strictly ergodic, provides a useful tool for, e.g., parameter estimation. The Adaptive Proposal algorithm is an automatic, easy to use and fast MCMC method. At least with reasonably well behaving unimodal target distributions, which we used for the comparison, the difference between the simulated distribution and the original target distribution is practically insignificant. The algorithm appears to be fairly insensitive with respect to the memory and update frequency parameters H and U. The Adaptive Proposal algorithm can also be used as an effective burn-in for ergodic MCMC methods.

Acknowledgments

The second author (E.S) has been supported by the Academy of Finland, Project 32837. We thank Elja Arjas, Esa Nummelin and Antti Penttinen for useful discussions on the topics of the paper.

References

Gelfand, A. E. & Smith, A. F. M. (1990), 'Sampling-based approaches to calculate marginal densities', *J. Am. Stat. Ass.* **85**, 853–409.

Gelman, A. G., Roberts, G. O. & Gilks, W. R. (1996), Efficient Metropolis jumping rules, *in* J. M. Bernardo, J. O. Berger, A. F. David & A. F. M. Smith, eds, 'Bayesian Statistics V', Oxford University press, Oxford, pp. 599–608.

Gilks, W., Best, N. & Tan, K. (1995), 'Adaptive rejection Metropolis sampling', *Applied Statistics* **44**, 455–472.

Gilks, W. R., Richardson, S. & Spiegelhalter, D. J. (1995), Introducing Markov chain Monte Carlo, *in* W. R. Gilks, S. Richardson & D. J. Spiegelhalter, eds, 'Markov Chain Monte Carlo in Practice', Chapman & Hall, pp. 1–19.

Gilks, W. R., Roberts, G. O. & George, E. I. (1994), 'Adaptive direction sampling', *Statistical Science* **43**, 179–189.

Gilks, W. & Roberts, G. (1995), Stategies for improving MCMC, *in* W. R. Gilks, S. Richardson & D. J. Spiegelhalter, eds, 'Markov Chain Monte Carlo in Practice', Chapman & Hall, pp. 75–88.

Gilks, W., Roberts, G. & Sahu, S. (1998), 'Adaptive Markov chain Monte Carlo through regeneration', *J. Amer. Statist. Assoc.* . To appear.

Gilks, W. & Wild, P. (1992), 'Adaptive rejection sampling for Gibbs sampling', *Applied Statistics* **41**, 337–348.

Haario, H., Saksman, E. & Tamminen, J. (1998a), Adaptive proposal distribution for random walk Metropolis algorithm. Reports of the Department of Mathematics, University of Helsinki, Preprint 176.

Haario, H., Saksman, E. & Tamminen, J. (1998b), An Adaptive Metropolis algorithm. Reports of the Department of Mathematics, University of Helsinki, Preprint 187.

Hastings, W. (1970), 'Monte Carlo sampling methods using Markov chains and their applications', *Biometrica* **57**, 97–109.

Kyrölä, E., Sihvola, E., Kotivuori, Y., Tikka, M., Tuomi, T. & Haario, H. (1993), 'Inverse Theory for Occultation Measurements, 1. Spectral Inversion', *J. Geophys. Res.* **98**, 7367–7381.

Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H. & Teller, E. (1953), 'Equations of state calculations by fast computing machine', *J. Chem. Phys.* **21**, 1087–1091.

Press, W. H., Teukolsky, S. A., Vetterling, W. T. & Flannery, B. P. (1992), *Numerical Recipes in FORTRAN, The art of scientific computing*, Clasendon Press, Oxford.

Roberts, G. O., Gelman, A. & Gilks, W. R. (1994), Weak convergence and optimal scaling of random walk metropolis algorithms. Preprint.
URL: *http://www.stats.bris.ac.uk/MCMC/*

Sahu, S. K. & Zhigljavsky, A. A. (1998a), Self regenerative Markov chain Monte Carlo. Preprint.
URL: *http://www.stats.bris.ac.uk/MCMC/*

Sahu, S. K. & Zhigljavsky, A. A. (1998b), Adaptation for self regenerative MCMC. Preprint.
URL: *http://www.stats.bris.ac.uk/MCMC/*